TECHNICAL DUE DILIGENCE REPORT

AutoTrader UK

Architecture, feature inventory, build & QA effort, risk

Subject: www.autotrader.co.uk (Auto Trader Group plc, LSE: AUTO)

Analysis performed 29 September 2026 · PDF compiled 14 October 2026

Based on public sources; live site returned HTTP 403 to direct inspection.

Contents

1. Executive Summary 3

2. Architecture Breakdown 3

3. Feature/Module Inventory 5

4. Build Effort Estimate — "Vibe Coding" Workflow 6

5. Testing & QA Time Estimate 8

6. Risk Factors / Estimate Blowup Triggers 9

7. Assumptions Made 10

Clarifying questions 10

Sources 11

AutoTrader UK (autotrader.co.uk) — Technical Due Diligence Report

> **Research method:** Direct inspection of the live site (`WebFetch`) was blocked — `https://www.autotrader.co.uk` returned **HTTP 403 Forbidden**, consistent with enterprise bot/WAF protection rather than a broken page. I could not read raw HTML, script bundle names, or response headers directly. Everything below is therefore built from **public corroborating sources** (Cloudflare and Google Cloud customer case studies, Auto Trader's own engineering blog/job postings, their PLC investor site, and Auto Trader Group's public profile) rather than page-source inspection. I've marked confidence levels throughout — treat anything marked Medium/Low as informed inference, not fact.

1. Executive Summary

AutoTrader UK is not a typical SaaS app or marketplace clone target — it's the **UK's largest online automotive marketplace**, operated by Auto Trader Group plc, a London Stock Exchange-listed company (LSE: AUTO, ~£800M+ annual revenue). It's a **two-sided marketplace**: free consumer-facing car search/browse/valuation for the public, monetized entirely through B2B subscriptions and advertising sold to ~13,000+ car dealers who list inventory and pay for the "Retailer" platform (stock management, valuations, analytics, advertising placement). Internally it runs on **600+ systems** with **~60,000 production releases/year**, per their own engineering blog.

> **Complexity tier: Very Complex** (top of scale — this is closer to "large enterprise marketplace platform" than "SaaS app," and full parity is not a solo/small-team undertaking regardless of tooling).

Because of that scale mismatch, this report gives two distinct things: (a) an architecture/feature breakdown of the real platform, and (b) a build-effort estimate for a **realistic, scoped-down MVP clone of the core consumer experience** — not full production parity, which isn't meaningfully estimable in hours (see §4 and §7).

2. Architecture Breakdown

Layer	Assessment	Confidence
Frontend	Mixed **React** and **Angular**, with **TypeScript**, across different product surfaces (typical of a large org where newer surfaces are React and older/dealer-tooling surfaces are legacy Angular)	Medium — from job postings/StackShare, not confirmed by page source since direct fetch was blocked
Rendering strategy	Unknown directly; a consumer marketplace at this scale and with this much SEO dependency (car listings are heavily search-indexed) almost certainly uses **SSR or hybrid SSR/CSR** for listing pages, with CSR for interactive tools (valuations, filters, saved searches)	Low-Medium (inferred from product shape, not confirmed)

Layer	Assessment	Confidence
Backend/API	**Java and Kotlin** on **Spring Boot**, **microservices architecture**, **Apache Kafka** for event-driven/async communication between services	High — stated directly in Auto Trader's own job postings and engineering blog summaries
API style	REST is the baseline; large orgs of this shape frequently layer GraphQL as an aggregation gateway over microservices, but I found no direct confirmation this applies here	Low — inferred, not confirmed
Database	Mix of **relational (SQL)** and **MongoDB** (document store), consistent with a system that has both highly structured transactional data (accounts, billing, dealer records) and semi-structured/variable data (vehicle listings, which vary wildly by make/model/spec)	Medium
Compute/hosting	**Google Cloud Platform**, specifically **Google Kubernetes Engine (GKE)** — confirmed via a published Google Cloud customer case study	High
CDN / DNS / WAF / Edge security	**Cloudflare** — confirmed via a Cloudflare customer case study/interview describing Auto Trader's migration of 9,000+ dealer zones and their main autotrader.co.uk zone to Cloudflare for DNS, WAF, and edge performance (cut DNS latency from ~20-25ms to ~10-13ms)	High
The 403 I hit	Consistent with Cloudflare Bot Management or similar anti-scraping sitting in front of the site — itself a signal this platform actively defends against automated inventory scraping (dealer listing data is commercially valuable)	Medium (plausible explanation, not confirmed root cause)
Auth model	Not directly observed. A marketplace with consumer saved-searches/alerts + a separate B2B dealer portal ("Auto Trader Retailer") almost certainly runs **two separate auth systems**: lightweight consumer account (email/password or social login, low friction) and a more permissioned **dealer/business account** system (multi-user per dealership, role-based access for stock managers vs. owners)	Low-Medium (inferred from product structure)
Notable technical features	Real-time-ish dealer stock sync from thousands of external dealer inventory/DMS feeds; a **proprietary valuation engine** (ML-based "Trended Valuations" — retail/trade/part-exchange pricing, trained on transaction data from 500,000+ daily listings, 3,500+ dealer websites, and auction/fleet disposal prices); large-scale image handling (millions of vehicle photos); marketplace-scale search/filter/ranking; alerting/notification pipeline (saved search → email/push)	High for *existence* of these features (confirmed via press releases/product pages), Low for *implementation detail*

3. Feature/Module Inventory

Visible/confirmed features (from product pages, press releases, help docs):

- Car search/browse with filtering (make, model, price, mileage, location, etc.)
- Listing detail pages (photos, spec, price, seller info, "price indicator" rating)
- Saved searches + alerts
- **Valuations tool**: Retail / Trade / Part-exchange valuations, plus "Trended Valuations" (6-month historical + 6-month forecast pricing)
- Part-exchange flow ("value your car, then browse")
- Dealer reviews/ratings
- **Auto Trader Retailer** platform (B2B): stock management dashboard, pricing/repricing suggestions, local/national retail rating comparisons, competitor stock comparison
- Finance/lead generation flow (car finance calculators, credit broker introductions — this implies FCA consumer credit regulatory scope, see §6)
- Vehicle history/check product (HPI-style)
- Native iOS/Android consumer apps (referenced in their product ecosystem)
- Contact-seller / lead capture forms
- Editorial content (reviews, buying guides — typical of large auto marketplaces, drives SEO)

Implied backend features (must exist, not directly visible):

- Ingestion pipeline normalizing inventory feeds from thousands of heterogeneous dealer DMS/feed formats
- Fraud/spam detection for listings (a marketplace this size is a constant scraping/fraud target)
- Image processing/optimization pipeline at scale (resize, CDN distribution, moderation)
- ML training/serving pipeline for the valuation engine (feature store, retraining cadence, drift monitoring)
- Billing/subscription system for ~13,000+ dealer accounts (tiered packages, usage-based ad placement)
- Event bus (Kafka) wiring together the 600+ systems — service discovery, schema management, dead-letter handling
- Search indexing/relevance ranking infrastructure (likely Elasticsearch/Solr-class, unconfirmed)
- Notification/alerting delivery (email/push at consumer scale)
- Audit logging, especially around dealer billing and FCA-regulated finance-broking activity
- CI/CD supporting ~60,000 deploys/year across 600+ systems — strongly implies mature internal platform/DevOps tooling, feature flagging, canary/rollback infrastructure
- Multi-tenant permission model for dealer group accounts (a dealer group can own many "Retailer" seats)

4. Build Effort Estimate — "Vibe Coding" Workflow

> **Important scoping note:** Full production parity with the real AutoTrader (600+ systems, a proprietary valuation model trained on years of transaction data across 3,500+ dealer feeds, dealer billing at scale, FCA-regulated finance broking, fraud defenses tuned against real attackers) is **not something I can responsibly put a number on** — that's a multi-year, hundreds-of-engineers undertaking, and no amount of AI-assisted velocity changes that, because the bottleneck isn't code-typing speed, it's **proprietary data you don't have** and **organizational/regulatory processes**, not something an agent can vibe-code around.

What follows is a defensible estimate for a **realistic MVP clone of the core consumer-facing marketplace experience** — the kind of scope you'd build to demo/prototype/portfolio-piece the product concept, using seeded/sample data instead of real dealer integrations, and a third-party valuation API stub instead of a proprietary ML model.

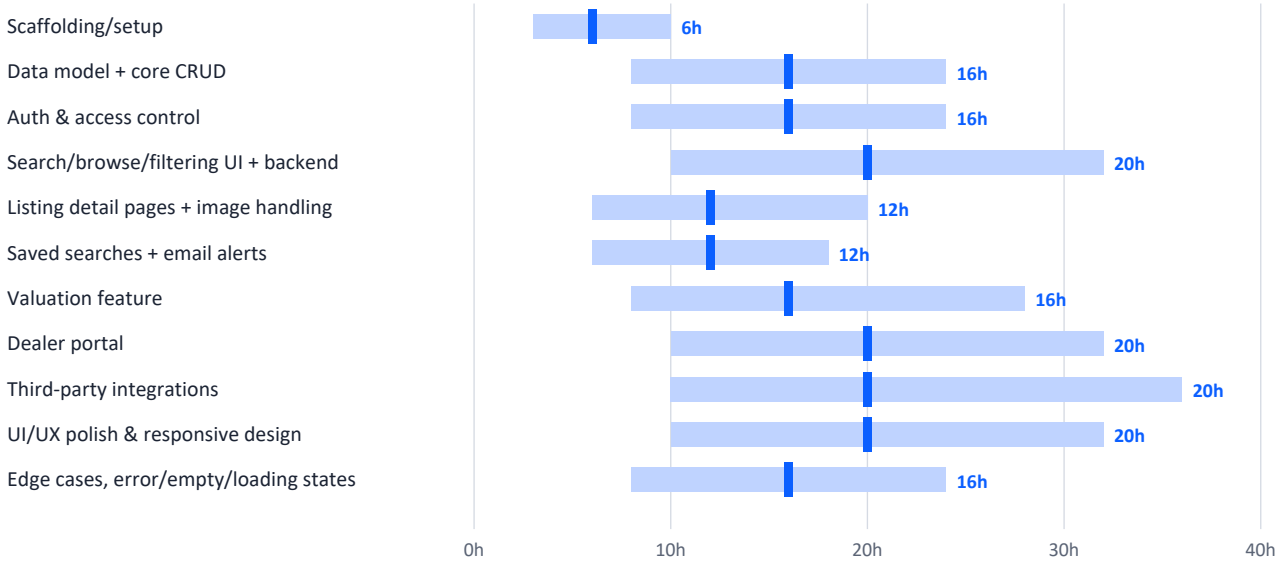
In scope: browse/search listings, listing detail pages, consumer accounts, saved searches + email alerts, basic seller/dealer account with manual stock CRUD (not real multi-feed ingestion), a valuation estimate via a bolted-on third-party API (not built in-house), contact-seller lead forms, responsive polished UI.

Out of scope (not estimated): real dealer feed ingestion at scale, proprietary ML valuation engine, FCA-regulated finance broking, native mobile apps, fraud/anti-scraping systems, 600-system-scale infra.

Phase	Optimistic	Realistic	Pessimistic	Notes
Scaffolding/setup (repo, stack, CI, dev env)	3h	6h	10h	Fast with AI — standard boilerplate
Data model + core CRUD (listings, users, dealers)	8h	16h	24h	Vehicle spec data model has real variability (make/model/trim/options) — schema design needs care even if CRUD is fast
Auth & access control (consumer + separate dealer roles)	8h	16h	24h	Two distinct auth surfaces; role separation is easy to get subtly wrong
Search/browse/filtering UI + backend	10h	20h	32h	Filter combinatorics + pagination/perf on a large seeded dataset; AI-fast for UI, slower for query/index tuning
Listing detail pages + image handling	6h	12h	20h	Multi-image galleries, responsive — mostly UI, fast
Saved searches + email alerts	6h	12h	18h	Background job + email delivery; moderate
Valuation feature (via third-party API, not in-house ML)	8h	16h	28h	**Integration bottleneck** — picking/wiring a real valuation data API (e.g. a UK vehicle valuation API), handling missing-VIN/no-match cases
Dealer portal (basic stock CRUD, no real feed ingestion)	10h	20h	32h	Multi-tenant dashboard, permissioning

Phase	Optimistic	Realistic	Pessimistic	Notes
Third-party integrations (email/SMS, payments if dealer billing is in scope, maps/location)	10h	20h	36h	**Flagged as the AI-velocity bottleneck** — webhook edge cases, sandbox-vs-live quirks, auth token lifecycle
UI/UX polish & responsive design (matching AutoTrader's visual bar)	10h	20h	32h	This site has a high design bar; getting "looks legit" takes real iteration even with AI
Edge cases, error/empty/loading states	8h	16h	24h	Easy to underestimate; large surface area (search, alerts, dealer CRUD all need these)
Total	**~87h (~2.2 wks)**	**~174h (~4.3 wks)**	**~280h (~7 wks)**	Full-time solo dev, AI-assisted

Effort range per phase (hours) — bar = optimistic → pessimistic, marker = realistic



Where AI-assisted coding is fast: CRUD scaffolding, list/detail UI components, form handling, standard REST endpoints, responsive layout, boilerplate auth (login/register/session).

Where it's slow/risky even in a vibe-coding workflow:

- **Valuation logic** — the real AutoTrader's moat is proprietary pricing data; any clone's valuation feature is only as good as whatever third-party API you bolt on, and reconciling its edge cases (no VIN match, sparse-data vehicles) eats real human review time.
- **Dealer billing/subscriptions** if in scope — Stripe-style webhook race conditions and plan-limit enforcement need careful human review, not just prompting (this is a known sharp edge even in much smaller apps — see this repo's own `mvp_addition_9` plan-gating design for a small-scale example of how much care that requires).
- **Search relevance/filtering at scale** — trivial to get "working," hard to get "good," and AI won't intuit what "good" means for car-buyer search behavior without real usage data.
- **Two-sided marketplace cold start** isn't a coding problem at all — no amount of build velocity fixes "no dealers means no inventory means no product."

5. Testing & QA Time Estimate

Scoped to the same MVP clone from §4.

Area	Estimate	Notes
Manual test case writing	~60-90 cases	Search/filter combinations, listing CRUD, saved-search/alert delivery, dealer stock CRUD, valuation edge cases (no match, partial data), auth boundaries
Functional/regression pass	12-20h	
Cross-browser & responsive/mobile	8-14h	Car-shopping traffic skews heavily mobile — this matters more than average
Integration/webhook testing (valuation API, email delivery, payments if in scope)	10-18h	Third-party failure modes: API timeout, rate limit, malformed response, webhook retry/dedup
Auth & access-control testing	8-14h	Consumer vs. dealer role boundaries; dealer-A-sees-dealer-B's-stock is the class of bug to specifically hunt for in a multi-tenant model
Security basics (input validation, injection, exposed endpoints)	6-10h	Standard OWASP pass
Load/performance testing	8-16h (only if you're actually claiming marketplace-scale traffic)	Not warranted for a portfolio-scale clone; warranted if this is meant to handle real dealer traffic
Total QA range	**~50-90h solo**, or **~35-60h elapsed with a dedicated QA person working in parallel with dev**	Solo dev doing both build and QA serially is the realistic default; a second QA person mostly buys wall-clock time, not less total effort

6. Risk Factors / Estimate Blowup Triggers

- **Proprietary data moat**: the real product's valuation accuracy comes from years of transaction data across hundreds of thousands of listings — a clone's valuation feature is structurally weaker no matter how much dev time is spent, unless you license comparable data.
- **Two-sided marketplace cold-start**: no coding amount solves "need dealers to get inventory, need inventory to get consumers, need consumers to get dealers."
- **FCA regulation**: AutoTrader brokers car finance leads, which puts them in UK consumer-credit-broking regulatory scope. Any clone that touches finance/credit introduction (not just search/browse) inherits real compliance obligations, not just engineering ones.
- **PCI scope** if dealer billing/subscriptions are handled directly rather than via a compliant processor.
- **Scale of dealer feed ingestion**: real-world dealer inventory feeds are messy, heterogeneous, and constantly changing formats — this is a long-tail integration problem, not a one-time build.
- **Anti-fraud/anti-scraping**: a public car marketplace is a high-value scraping target (competitors, data brokers); the 403 I hit while researching this is itself a small data point that this is actively defended in the real product.
- **Native mobile apps**: not included in the estimate above; if required, add a comparable multi-week effort per platform (or cross-platform framework time) plus app-store review cycles.
- **SEO/content scale**: millions of indexable listing pages is itself an infra/performance consideration (crawl budget, sitemap generation, stale-listing handling) beyond typical CRUD-app concerns.

7. Assumptions Made

- Direct page-source inspection was unavailable (403 on fetch), so frontend framework, exact rendering strategy, and API style are inferred from secondary sources, not confirmed firsthand — flagged at Medium/Low confidence above.
- The build/QA estimates in §4-5 scope to a **consumer-facing MVP clone**, explicitly excluding real dealer-feed integration, proprietary ML valuation, FCA-regulated finance broking, native mobile, and marketplace-scale anti-fraud — because those aren't meaningfully estimable in "vibe coding hours."
- Assumes one experienced solo developer, full-time, using an AI coding agent (Claude Code/Cursor-class tooling).
- Assumes a seeded/sample dataset for vehicle listings rather than real dealer inventory at launch.
- Assumes standard third-party services (a generic vehicle valuation API, standard email provider) rather than building proprietary data pipelines.
- Assumes no specific compliance requirement (FCA, PCI) unless the finance-broking or payments features are explicitly pulled into scope.

Clarifying questions

1. Is the goal a **portfolio/prototype-scale clone** of the consumer browsing + valuation experience (what §4/§5 estimate), or are you actually trying to scope **competing with the real platform** (dealer network, proprietary valuations, FCA compliance) — because those are different projects by orders of magnitude, not different phases of the same one?
2. Does the dealer-side "Retailer" portal (stock management, billing, analytics) need to be in scope, or is this consumer-search-only?
3. Are native iOS/Android apps required, or is a responsive web app sufficient?
4. Is real payment/subscription billing (dealer plans) or finance-lead brokering in scope — this materially changes the compliance/QA burden (§6)?

Sources

- Autotrader Engineering (Medium)
- About Autotrader Engineering
- Senior Full Stack Engineer job posting — Auto Trader
- auto trader — StackShare
- Auto Trader (UK) Case Study — Google Cloud
- Auto Trader Case Study — Google Cloud
- Why AutoTrader Migrated DNS and WAF to Cloudflare — Cloudflare TV
- Auto Trader UK — Cloudflare Case Study
- Auto Trader launches trended valuations
- Part Exchange Your Car with Autotrader
- Autotrader Valuations | Autotrader
- Auto Trader gives dealers access to valuations tool — Motor Trader
- 2025 Annual Report — Auto Trader plc Investors